ZIV · JUDGE REPRO GUIDE

Every claim, with a way to check it.

A reproducible guide to the Ziv haptic message channel: the hosted feel-tool, the hermetic test run, and the live relay checks. No phone, no provider key.

232
hermetic tests passed
5 browser e2e deselected

274
firmware bench checks
0 failures

49
host demo checks
27 HAP events matched

76.2s
current demo cut
Kore narration, captions

Nebius x NVIDIA Global AI Hackathon · Best Apps and Agents
Submitted 28 October 2026 · github.com/razukc/ziv
Video: youtu.be/t9ovgEjm8i8

Every testable behavioral claim in this guide maps to a test, a command, or a captured live-origin response. Three ways in: the **hosted feel-tool** (interactive timing source, no install), the **~2.5-minute hermetic test run** (no phone, no key), and the **live run-through** (real server, real restart, real HTTP). Nothing here needs `NEBIUS_API_KEY`; the model path is proven at the wire contract level with a mocked provider, so the turn/gate/queue/close pipeline is exercised without a live provider call. With a key and a funded account, post an `intent` instead of a `text` on the same schedule call: the relay makes the real call when the reminder fires and the turn is labelled `text_live` (`text_echo` when no key is set, and also when the provider refuses).

The public origin is keyed: its `.env` carries a funded Token Factory key, the model is pinned, and `text.last_compose` records the last real call — so the same intent can be driven end-to-end by hand, with no key of your own (the console also takes your key for a one-shot call; nothing is stored).

The **hosted feel-tool** is at https://site-5505fea77ec84b4f8e1a5d70c82d5877.freebuff.page: the interactive version of the generated timing source and attention vocabulary, with name-mark cards, the spell box, Session mode (M1), and blind A/B. No install, no key; Android Chrome vibrates, desktop and iOS show the visual rhythm only.

Quick index — claim → where it's proven:

Claim	Proof
Hosted feel-tool: name marks, attention vocabulary, spell box, M1/A-B	https://site-5505fea77ec84b4f8e1a5d70c82d5877.freebuff.page (source: `docs/haptic-name-marks.html`; generated from `docs/haptic-timing.json`; drift guard: `tests/test_haptic_timing.py`)
Turn journey order on the wire	`tests/test_ziv_server.py::test_turn_journey_reaches_the_wire_in_order`
Queue-don't-interrupt + 429	`test_message_is_rejected_429_when_replay_queue_is_full`
Error long-buzz on provider failure	`test_omni_provider_failure_feels_the_error_long_buzz`
Z-I-V name mark + triple-pulse tail	`test_prefix_mark_order_is_mark_tail_then_turn`
Live fire loop fires reminders	`test_scheduled_reminder_fires_with_name_mark_over_the_wire`
Schedule survives restarts (durable)	`tests/test_ziv_store.py::test_schedule_roundtrip_across_instances`
Schedule: claim-before-play, one clock, corrupt recovery	`test_schedule_take_due_claims_then_releases`, `test_schedule_take_due_uses_one_now`, `test_schedule_clear_and_corrupt_recovery`
Concurrent arrivals: no loss, no dup, loud refusals	`tests/test_gate_concurrency.py` (3 tests: `test_concurrent_arrivals_never_lose_or_duplicate`, `test_every_message_is_accounted_exactly_once`, `test_refusals_are_loud_and_bounded`)
Bursts queue instead of cutting in, and the cap refuses loudly	`test_burst_of_arrivals_plays_one_and_bounds_the_rest`, `test_seam_gate_stampede_queues_every_arrival_and_no_loss`, `test_seam_gate_stampede_beyond_cap_rejects_and_never_queues_overflow`
An arrival during the model wait queues instead of barging	`test_arrival_during_the_model_wait_queues_instead_of_barging`

Claim	Proof
A band vanishing mid-turn frees the channel and keeps the text	`test_band_vanishing_mid_turn_releases_the_gate`, `test_band_vanishing_mid_turn_stores_the_in_flight_text`, `test_band_vanishing_mid_turn_strands_queued_messages_not_drops_them`
A dying turn cannot release the *next* arrival's channel	`test_an_aborting_turn_does_not_release_the_next_arrival_s_channel`
Inbox replay is a self-naming turn	`test_reminder_stored_offline_replays_with_name_mark`
A stored backlog never strands what cued behind it	`test_inbox_delivery_drains_what_arrived_behind_it`, `test_inbox_delivery_strands_what_it_could_not_play`, `test_inbox_delivery_declines_while_a_turn_holds_the_channel`
The wearer's pace is a real dwell, not a stored number	`test_the_wearer_pace_lands_on_the_wire_as_real_dwells`, `test_moving_the_pace_moves_every_dwell_by_exactly_that_much`
Auth covers every route that carries or reveals content	`test_auth_covers_every_route_that_carries_or_reveals_content` (8 method/path pairs), `test_auth_leaves_the_bootstrap_routes_open`
PWA badge contract (both sides pinned)	`tests/test_client_badge_contract.py` + `test_health_payload_serves_the_pwa_badge_contract`
Manifest name/description match the gallery cover	`tests/test_client_manifest_contract.py` (`name` ← cover title, `description` ← cover caption; both read from `media/make_gallery.py`)
Health surfaces the schedule (depth/cap/corrupt)	`test_health_payload_surfaces_the_schedule`
Redial give-up is a loud dead end	`tests/test_client_redial_contract.py` (hermetic) + `test_redial_giveup_e2e.py` (browser; run with `-m e2e`)
Timing spec: one source, drift-guarded	`tests/test_haptic_timing.py`, `test_rename_check.py`
Firmware bench: the sequencer plays the spec in host-portable C	`firmware/haptic_out/run_tests.py` → **274 checks, 0 failures** (`-Wall -Wextra -Werror`, no ESP-IDF needed)
Host demo (rung 1): the ESP32-S3 app core's scripted boot demo plays the derived fixture exactly	`firmware/app/ziv_qemu/run_ziv_tests.py` → **49 checks, 0 failures** — 27 HAP events matched within tolerance (start times, payload names, dot masks, buzz/gap, beat index). This is the app core compiled for the host; a QEMU boot is still pending an IDF install (`docs/QEMU_SIMULATION_LADDER.md`).
Honesty audit trail	`HONESTY_CHANGELOG.md`

> **Reproducibility in this checkout.** This checkout contains everything the rows above need: all 19 test modules plus `conftest.py` under `agent/tests/`, the `firmware/` tree, the `docs/` timing spec, and the generated consumers. Every row is reproducible here. The firmware rows were re-run for this checkout: 274 checks and 49 checks, 0 failures, 27 HAP events matched.

Step 0 — the test run

Green on this checkout, no network and no key. The demo requirements do not pin `pytest`; install it in the same venv first (`pip install pytest`):

```
cd agent
python -m pytest -q
# -> 232 passed, 5 deselected in about 211 s (3:31), exit 0
```

That is every hermetic test in the tree: 232 in 16 modules, with the five browser tests in the other three modules deselected by their `e2e` marker. The store file alone is the quick read:

```
cd agent
python -m pytest -q tests/test_ziv_store.py
# -> 17 passed in under a second
```

Every test is armed with a **90 s watchdog** (stdlib `faulthandler`, in `agent/tests/conftest.py`). If a case deadlocks, the run ends non-zero with the hanging test's id and every thread's stack in `agent/tests/pytest-hang.txt`, instead of leaving a terminal frozen with no verdict. Override the bound with `ZIV_TEST_TIMEOUT_S=30` or `@pytest.mark.timeout(seconds)`; the dump file is removed after a clean run.

The firmware bench is host-portable and **is** in this checkout. Run it from the repo root; the runners find `cc` / `gcc` / `clang` / `zig cc` (or exit 2 with an install hint):

```
python firmware/haptic_out/run_tests.py
# -> 274 checks, 0 failures
python firmware/app/ziv_qemu/run_ziv_tests.py
# -> 49 checks, 0 failures; PASS — 27 HAP events matched within tolerance
```

On this Windows checkout neither PATH nor WSL had a C compiler, so both were run with the project venv interpreter that carries `ziglang`: `agent/venv/Scripts/python.exe firmware/haptic_out/run_tests.py` (same for the QEMU runner). On Linux CI, plain `python` with `gcc` is enough.

Step 1 — the live run-through (live origin)

Nothing to check out, no venv, no key. Open **https://35-224-56-20.sslip.io** — tap **Open the band** — and drive it with the commands below from any shell.

Drive a turn by hand first; this is the request/response shape every other claim rides on:

```
curl -X POST https://35-224-56-20.sslip.io/api/ziv/message \
  -H 'Content-Type: application/json' -d '{"text": "port closed"}'
# -> {"event":"turn_complete","seconds":9.6,
#     "events":["message:play","attention:processing","attention:processing",
#               "message:play","lifecycle:playback_done"],"drained":0}
```

`seconds` is that run's pace (the model stub is 3 s plus one dwell per character), so treat it as approximate. If a scheduled reminder happens to be due while you post, the same response carries `"drained": 1`; the reminder then plays as its own turn after this close, not as part of this turn's `events` list.

Now a reminder nobody asked for (leave the band awake for this one):

```
NOW=$(date +%s)
curl -X POST https://35-224-56-20.sslip.io/api/ziv/schedule \
  -H 'Content-Type: application/json' \
  -d "{\"fire_at\": $((NOW+10)), \"text\": \"demo beat\"}"
# -> {"scheduled":true,"id":1,"fire_at":1792905801.0,"text":"demo beat",
#     "source":"scheduled","fire_at_iso":"2026-10-25T05:23:21Z"}
# id is whatever the store assigns; prior schedules make it higher.
```

The live origin's health reports `text.key_set: true`, but a `text` turn never calls the model. To exercise the agent path, use the same schedule call with `"intent": "meds at nine"` in place of `"text"`: the relay composes the line when it fires if the key is funded, and otherwise fires the intent verbatim as `text_echo`.

Read `text.configured` for "a key is wired" and `text.available` for proof: it is `null` until a real compose runs, then mirrors `last_compose.ok`, so an unfunded key now reports `available: false, last_compose.status: 402` instead of `available: true`.

As of 2026-10-27 the demo VM's key has trial credit and the relay pins `nvidia/NVIDIA-Nemotron-3-Nano-30B-A3B` with thinking disabled, so an intent is a real call.

The VM now runs the cap fix (HONESTY_CHANGELOG §1.12), redeployed 2026-10-27 18:06 UTC. Live, with a band attached: the intent `tell me to drink` composed **"drink water"** — 10 cells, all of them played — and health recorded `text.last_compose {ok: true, status: 200}`, `text.available: true`. Before the fix the same intent produced "drink water now" (13 letters) and the player silently dropped its final `w`.

On 2026-10-28 the demo take used the scheduled intent `meds at nine`; the live model wrote **"meds nine"** (8 cells), and health recorded `text.last_compose {ok: true, status: 200}` with `text.available: true`. That is the take in the 76.2-second cut (source MP4 `media/tts_audition/demo_kore/demo_kore.mp4`, gitignored). The earlier `tell me to drink` evidence above is kept as history, not as the current demo script.

The same call from Python, if shell quoting is fighting you:

```
import json, time, urllib.request
body = json.dumps({"fire_at": time.time() + 10, "text": "demo beat"}).encode()
req = urllib.request.Request("https://35-224-56-20.sslip.io/api/ziv/schedule",
                             data=body, headers={"Content-Type": "application/json"},
                             method="POST")
print(urllib.request.urlopen(req).read().decode())
```

Watch the awake band: `reminder: demo beat` arrives on its own as a self-naming turn — Z-I-V mark cells, then one cell per non-space character (11 cells for `ziv` + `demo beat`), then the close. Nothing on the page asked for it.

Health is the operator's view — badge numbers, caps, queue rejections, telemetry:

```
curl https://35-224-56-20.sslip.io/api/ziv/health
```

Look for `gate_queue` / `gate_queue_cap` (what the PWA badge renders), `queue_rejections.per_minute`, `schedule_pending` / `schedule_cap` / `schedule_corrupt`, `inbox_pending`, `store_problems`. The badge on the page polls the same endpoint every 2 s.

Cleanup: `curl -X DELETE https://35-224-56-20.sslip.io/api/ziv/schedule` (answer `{"cleared": 0}` if the reminder already fired — firing consumes it).

That origin is this repo's relay on a GCP VM behind Caddy (Let's Encrypt); the units and reboot evidence are in `agent/VM_ONLINE.md`. Both the relay and Caddy are systemd units, so a reboot costs about 20 s and nothing manual, and the URL is stable across restarts (reserved static IP; no domain of our own). It is deliberately unauthenticated for judging; please use it one at a time.

Step 2 — the durability beat (local, because it needs the relay killed)

The one claim a shared origin cannot demonstrate for you is "the schedule survives a restart" — nobody is going to stop the demo VM mid-judging. Run this part locally; every output below was captured live on this repo. Set the agent up per the README quick start first (that block builds the venv and installs the deps); the commands here run from the repo root and `cd agent` themselves.

If port `8787` is already taken on your machine, pick another with `ZIV_PORT=8790 python ziv_server.py` and use that port in every URL below.

```
cd agent
ZIV_PORT=8787 python ziv_server.py        # terminal 1
```

1. Schedule a reminder (10 s out) — beat 1, "the relay acts while the phone is away":

```
NOW=$(date +%s)
curl -X POST http://127.0.0.1:8787/api/ziv/schedule \
  -H 'Content-Type: application/json' \
  -d "{\"fire_at\": $((NOW+10)), \"text\": \"pills\"}"
# → {"scheduled":true,"id":1,...,"text":"pills","source":"scheduled",...}
```

2. Watch it fire unprompted, name-mark first — open the dev band in a browser: `http://127.0.0.1:8787/ziv_client/index.html`, tap **Open the band**, then feel/read the wire log: `reminder: pills` → Z-I-V cells → `triple-pulse` → breath → kind cue → processing ticks → cells → close. (No phone? The wire log on the page shows the same journey; `reminder: pills` arrives on its own.)

3. The durability beat — kill the relay, restart, the reminder survives:

A reminder 10 s out has already fired (and been consumed) by the time you get back to the shell, so this beat needs one that is still pending. Wait until the wire log shows its close before scheduling the next one — otherwise the list below has two rows. Schedule it far enough ahead:

```
curl -X POST http://127.0.0.1:8787/api/ziv/schedule \
  -H 'Content-Type: application/json' \
  -d "{\"fire_at\": $(( $(date +%s) + 600 )), \"text\": \"judge repro beat\"}"
# -> {"scheduled":true,"id":...,"text":"judge repro beat","source":"scheduled",...}
```

Ctrl-C the server, start it again exactly as before, then list the schedule:

```
ZIV_PORT=8787 python ziv_server.py        # same terminal
curl http://127.0.0.1:8787/api/ziv/schedule
```

Verified against a real kill on this repo:

```
{"reminders":[{"id":1,"fire_at":1792907289.0,"text":"judge repro beat",
  "source":"scheduled","fire_at_iso":"2026-10-25T05:48:09Z","fired":false}]}
```

Same `id`, same `fire_at`, same text, still `fired: false`. (`id` is whatever the store assigns; a fresh `agent/ziv_data/` starts at 1, and if you already ran step 1 your second reminder is `id` 2 — the captured row above is from a fresh store.) The earlier draft of this step said "the SAME reminder comes back" after the 10-second one — that one is consumed when it fires, so listing then correctly returns `{"reminders":[]}`.

4. Fire-while-away (store, don't drop):

```
# with NO band attached (close the browser tab first):
curl -X POST http://127.0.0.1:8787/inject/audio \
  -H 'Content-Type: application/json' -d '{"simulate": "who is calling"}'
# → {"event":"inbox:stored","stored":true,...}
curl -X POST http://127.0.0.1:8787/api/ziv/message \
  -H 'Content-Type: application/json' -d '{"text": "stored while away"}'
# → {"event":"inbox:stored",...}  (same no-band contract for typed input)
```

Reattach the band: hello reports the pending count, and each replay arrives as a **self-naming turn** — mark first, content after (inbox replay claim).

5. Health — the operator's view (badge numbers, telemetry):

```
curl http://127.0.0.1:8787/api/ziv/health
```

Look for: `gate_queue` / `gate_queue_cap` (what the PWA badge renders), `queue_rejections.per_minute`, `schedule_pending` / `schedule_cap` / `schedule_corrupt`, `inbox_pending`, `store_problems`. The badge on the dev-band page shows the same numbers live, polled every 2 s.

Cleanup: `curl -X DELETE http://127.0.0.1:8787/api/ziv/schedule`, then Ctrl-C. Wearer state lives in `agent/ziv_data/` (gitignored); DELETE clears your own test schedule.

Step 3 — optional, 5 minutes: the band on a phone

Open **https://35-224-56-20.sslip.io** in Android Chrome. There is no same-WiFi step and no LAN address to look up: it is a real HTTPS origin, so the service worker registers and Chrome offers **Install app**. A plain `http://<lan-ip>:8787` can do neither — service workers and install prompts require a secure context. Vibration needs Android Chrome; desktop Chrome renders the wire log, cells box, badge and redial note, but cannot buzz. (The same page also runs locally: start the relay and open `http://127.0.0.1:8787/ziv_client/index.html`.)

1. Send a message → read the wire log top-down: arrival cue → `processing` ticks → cue-as-content → one `cell` frame per letter → `end-of-message` close. On a phone, every frame vibrates with the exact pattern the firmware would play (both derive from `docs/haptic-timing.json`).
2. While it plays, tap **Write a message** in the device bar → the busy sheet, type, and press Send. The second gets its cue only, queues, and replays after the close without stopping the first turn.
3. Keep sending until the badge shows `refusing` → the sender gets HTTP 429 and the page shows the armed-redial note. The deployed client includes the three-close budget, the red **dead-end note**, and the busy-sheet path; all are pinned by `tests/test_client_redial_contract.py` and `tests/test_client_busy_sheet_contract.py` and were verified on the live HTTPS origin on 2026-10-25.

The five `e2e`-marked tests drive the visible compose surfaces: `#btnWrite` → `#composeInput` → `#btnSendCompose` from the idle screen, and `#btnWriteDuring` → the busy-sheet → Send while a turn owns the screens. They are deselected from the 232-count hermetic run. With Playwright installed they run separately:

```
cd agent
python -m pytest -m e2e -q
# -> 5 passed, 232 deselected in about 75 s (1:15)
```

What each artifact is

File	Role
`agent/ziv_server.py`	the relay: WS pump, schedule fire loop, health, Omni seam
`agent/ziv_relay.py`	the seam: `MessageGate` + `TurnTimeline` (the invariants)
`agent/ziv_store.py`	durable state: inbox, memory, **schedule** (atomic JSON, capped, corrupt-tolerant)
`agent/ziv_client/index.html`	the PWA dev band: timing bootstrap, badge, redial
`tools/haptic_timing.py`	generates and verifies all 7 consumers from `docs/haptic-timing.json`; `tools/haptic_timing_gen.py` is one of the generated consumers
`HONESTY_CHANGELOG.md`	every corrected claim, test-caught bug, and soft spot